



湖南大学
HUNAN UNIVERSITY

第二章 高级数据结构

—— 湖南大学计算机学院 ——

引言



- 对于计算机而言，数组拥有最高的访问效率（无指针跳转、内存连续）。
- 本节所介绍的高级数据结构，就是如何巧妙地利用数组来模拟复杂逻辑，从而兼顾性能与功能。
- 课程目标：
 - 堆：如何在数组中快速定位并维护最大值/最小值？

目录

第一节

堆

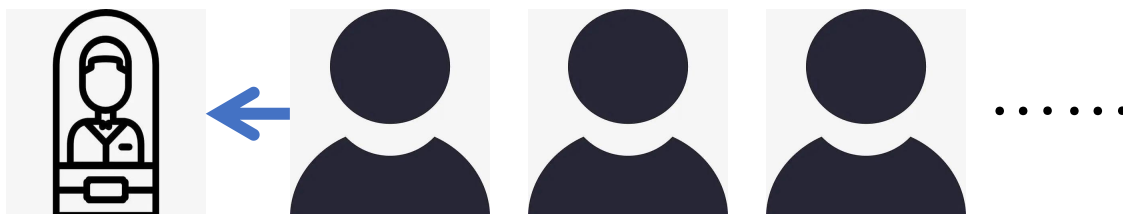
第二节

斐波拉契堆

队列



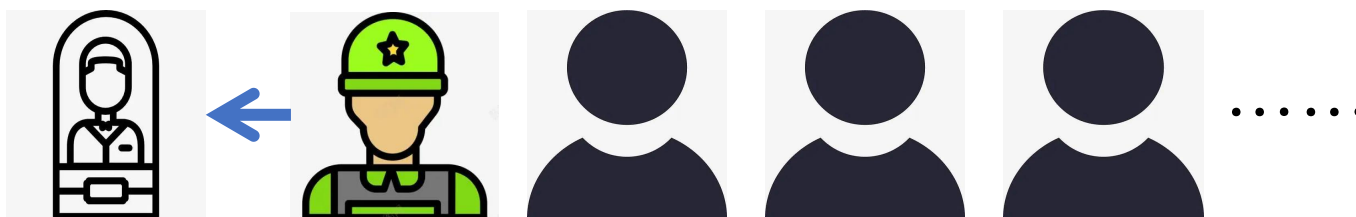
队列： 满足先进先出（FIFO）的数据结构，数据从队列头部取出，新的数据从队列尾部插入，数据之间是平等的。类似于普通人到火车站排队买票。



优先队列



优先队列：从头部取出数据，从尾部插入数据，但是这个过程根据数据的优先级而变化的，总是优先级高的先出来。类似于军人依法优先。



优先队列问题定义



优先队列是一种用来维护由一组元素构成的集合 S 的数据结构，其中每一个元素都有一个关键字(key)，关键字赋予了一个元素的优先级，故名为优先队列。

优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) $\text{Maximum}(S)$: 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) $\text{Extract_Max}(S)$: 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) $\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。

堆的作用



堆(Heap)是一类特殊的数据结构，是基于数组的高效的优先队列实现方式

堆的应用场景：

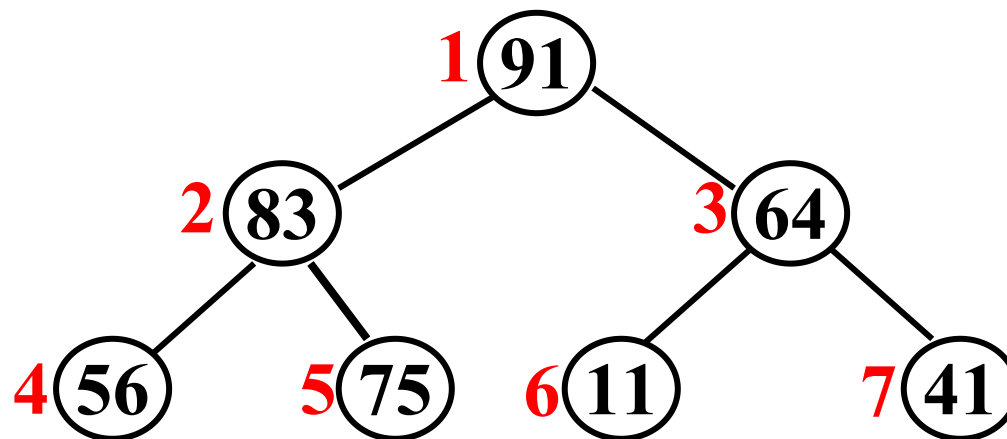
- 优先级队列的使用场景
- 求Top K值问题
- 求中位数、百分位数
- 大数据量日志统计搜索排行榜

课程目标： 堆的定义与操作

堆的定义



满二叉树：一棵深度为 k 且有 2^k-1 个结点的二叉树



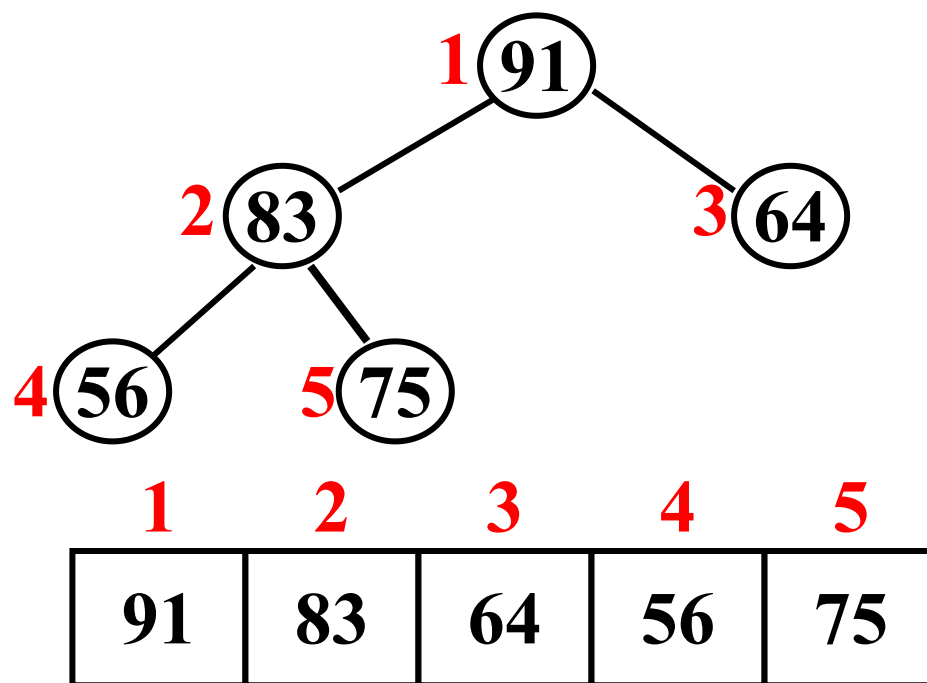
如果对满二叉树的结点进行编号, 约定编号从根结点起, 自上而下, 自左而右, 则可以形成一个长度为 2^k-1 的序列

1	2	3	4	5	6	7
91	83	64	56	75	11	41

堆的定义



完全二叉树：一棵深度为 k 的有 n 个结点的二叉树，对树中的结点按从上至下、从左到右的顺序进行编号，如果编号为 i ($1 \leq i \leq n$) 的结点与满二叉树中编号为 i 的结点在二叉树中的位置相同，则这棵二叉树称为完全二叉树。



堆的定义



堆实质上是满足如下性质的完全二叉树：树中任一非叶结点的关键字均不大于(或不小于)其左右孩子(若存在)结点的关键字

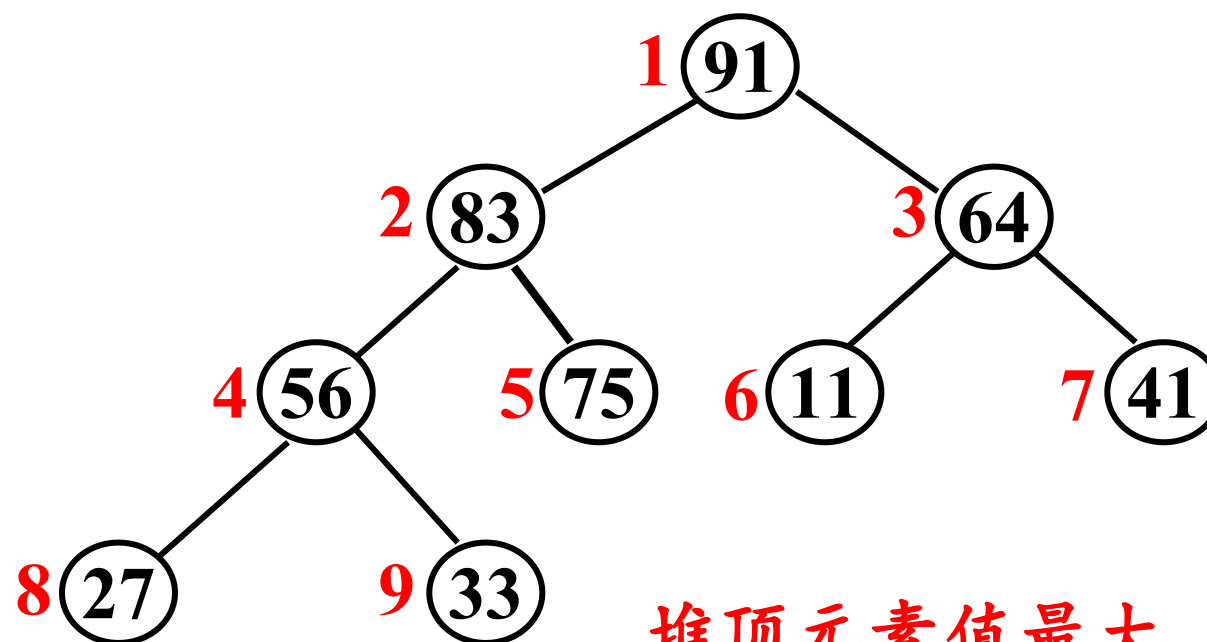
对应序列上， n 个元素的序列 $\{k_1, k_2, \dots, k_n\}$ 当且仅当满足如下关系时，称之为**堆 (heap)**。若满足条件 $k_i \leq k_{i/2}$ 且则称**最大堆**；若满足条件 $k_i \geq k_{i/2}$ 则称**最小堆**

堆的实例



最大堆

1	2	3	4	5	6	7	8	9
91	83	64	56	75	11	41	27	33



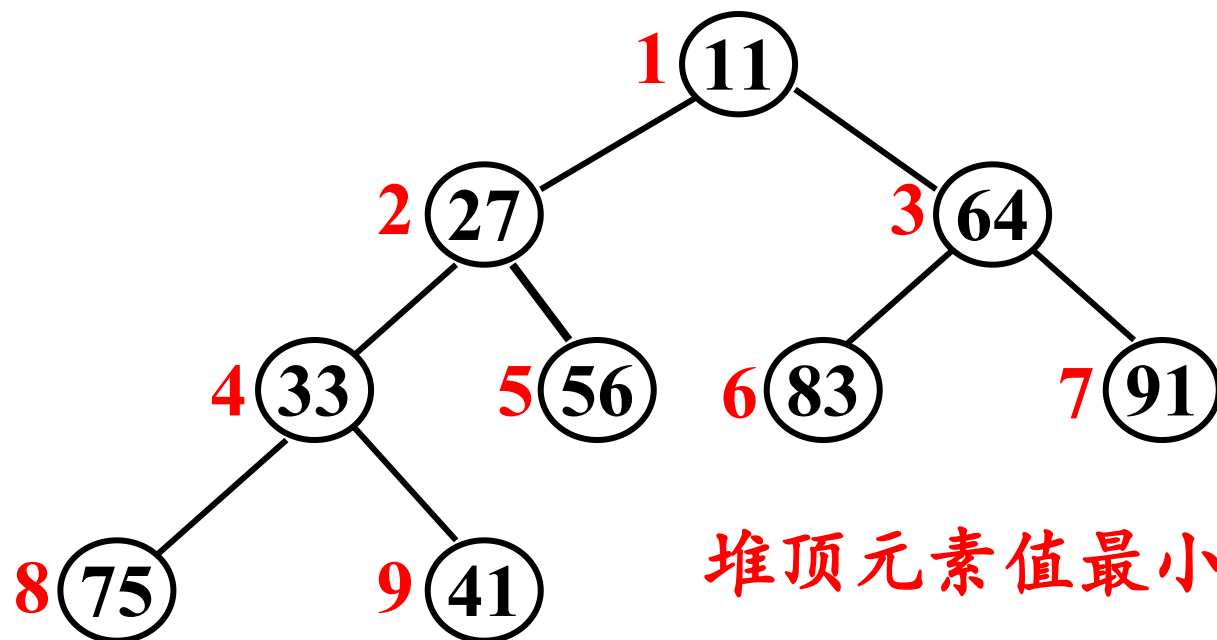
堆顶元素值最大

堆的实例



最小堆

1	2	3	4	5	6	7	8	9
11	27	64	33	56	83	91	75	41



堆顶元素值最小

将堆用顺序存储结构来
存储，则堆对应一组序
列

优先队列需求



优先队列的操作:

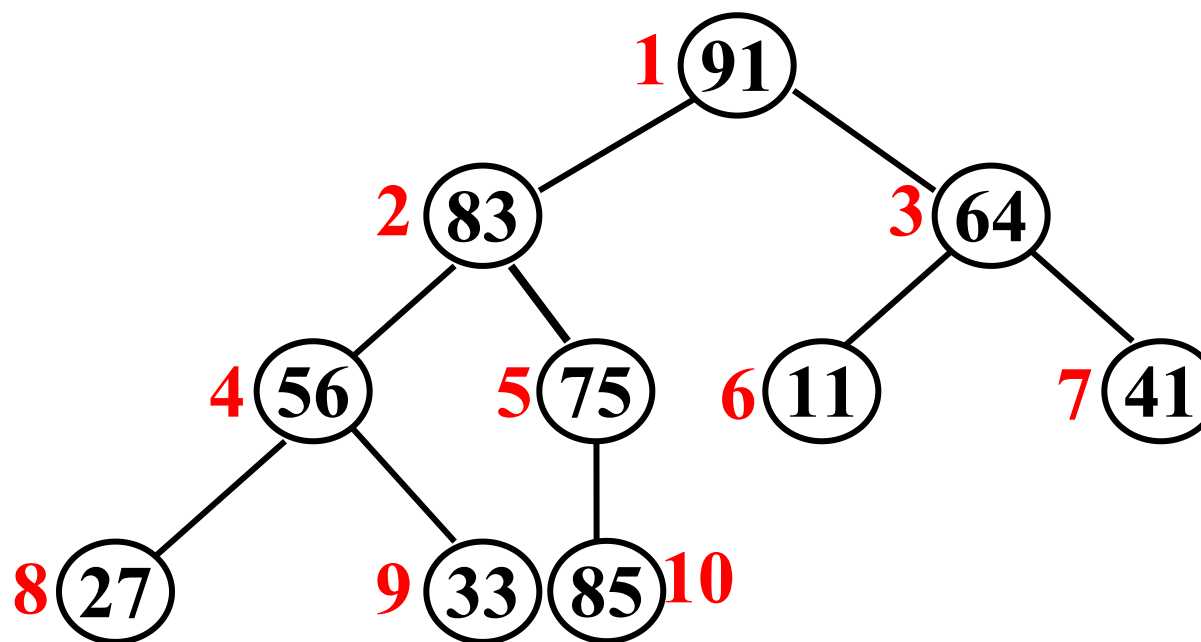
- 1) **Insert(S, x):**插入x到集合S中;
- 2) **Maximum(S):**返回S中具有最大 (或者最小) 关键字的元素;
- 3) **Extract_Max(S):**去掉并返回集合S中具有最大 (或者最小) 关键字的元素;
- 4) **Increase_Key(S, x, key):**将元素x的关键字增加到key。

堆的插入



插入一个新的元素85之后呢？

91	83	64	56	75	11	41	27	33	85			
----	----	----	----	----	----	----	----	----	----	--	--	--



堆的插入



如何在堆中插入一个元素？

解决方法：

在堆的最后增加一个元素，然后新添加的元素不能比它的父元素大，如果比它的父元素大，就涉及到上浮的动作

堆的插入



如何调整成一个堆?

91	83	64	56	75	11	41	27	33	85			
----	----	----	----	----	----	----	----	----	----	--	--	--

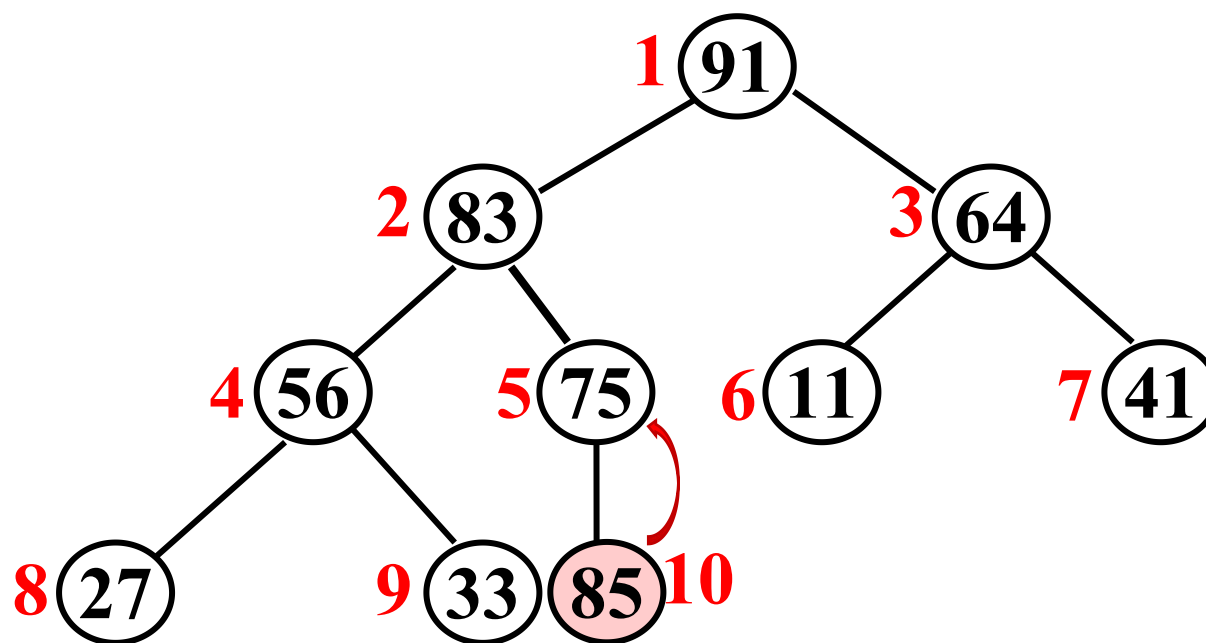
91	83	64	56	85	11	41	27	33	75			
----	----	----	----	----	----	----	----	----	----	--	--	--

91	85	64	56	83	11	41	27	33	75			
----	----	----	----	----	----	----	----	----	----	--	--	--

堆的插入



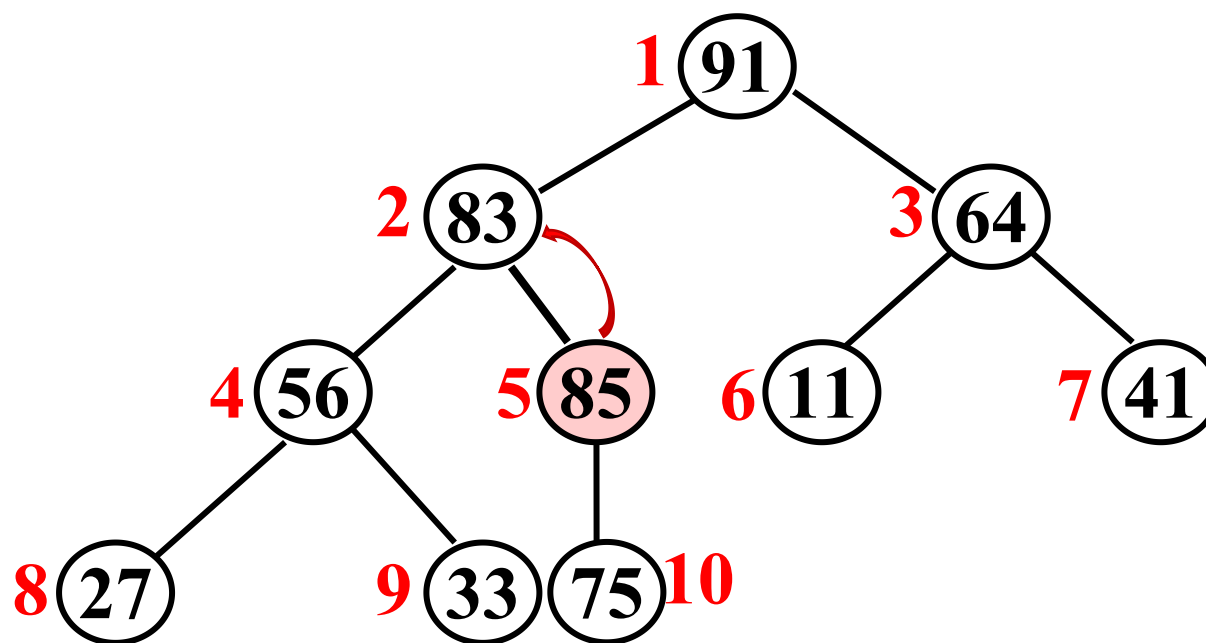
最大堆中加入元素85



堆的插入



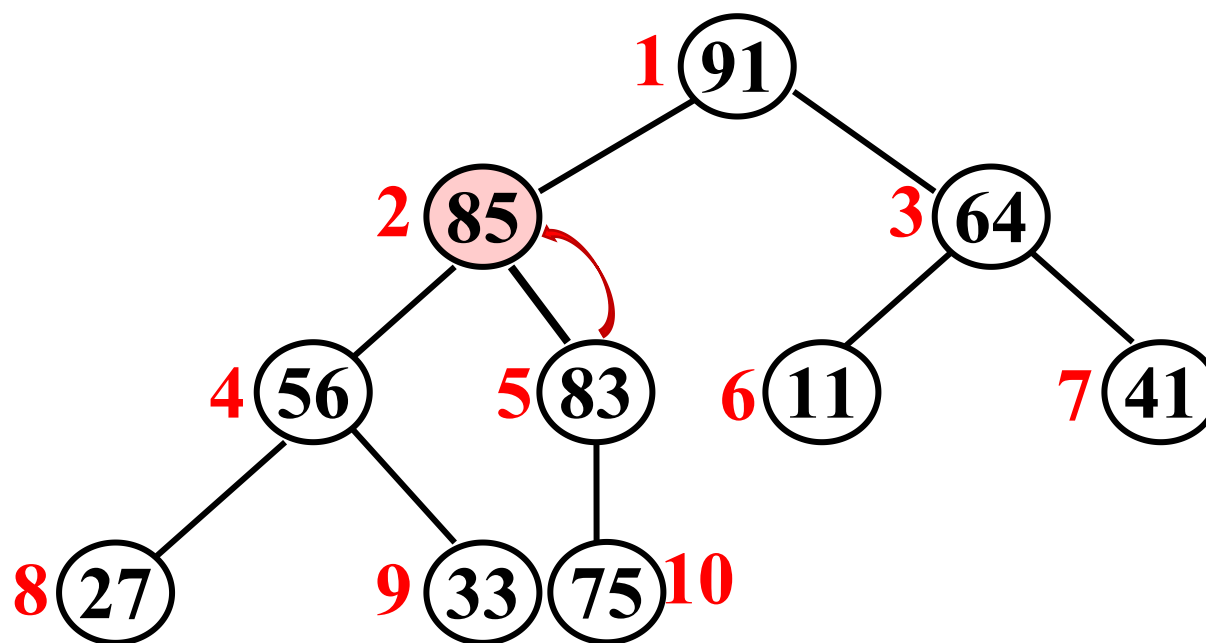
最大堆中加入元素85



堆的插入



最大堆中加入元素85



堆的插入



在堆中插入一个元素复杂度

最坏情况就是自底向上上浮到堆的根，于是时间代价为堆的深度 k

因为堆对应完全二叉树，而层数为 k 的完全二叉树最少有 2^{k-1}

于是，对于包含 n 个元素的堆而言

$$2^{k-1} \leq n \Rightarrow k - 1 \leq \log_2 n \Rightarrow k \leq \log_2 n + 1$$

因此，在堆中插入一个元素时间代价为 $O(\log n)$

优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) **$\text{Maximum}(S)$** : 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) **$\text{Extract_Max}(S)$** : 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) $\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。

堆的输出



如何从堆中输出最大（或者最小）元素？

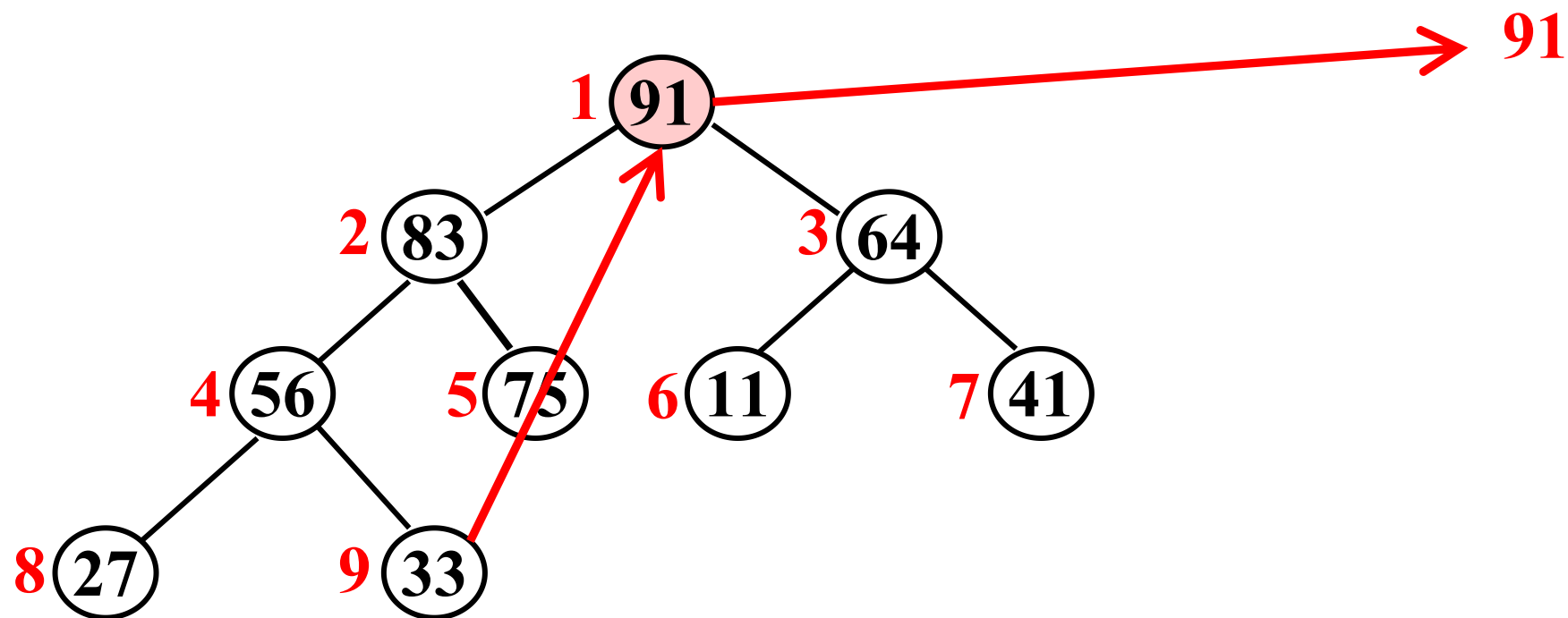
解决方法：

将最大（或者最小）元素输出，然后堆的最后元素放到堆的第一位，然后新的根元素不能比它的父元素小（或者大），如果比它的父元素小（或者大），就涉及到下沉的动作

堆的输出



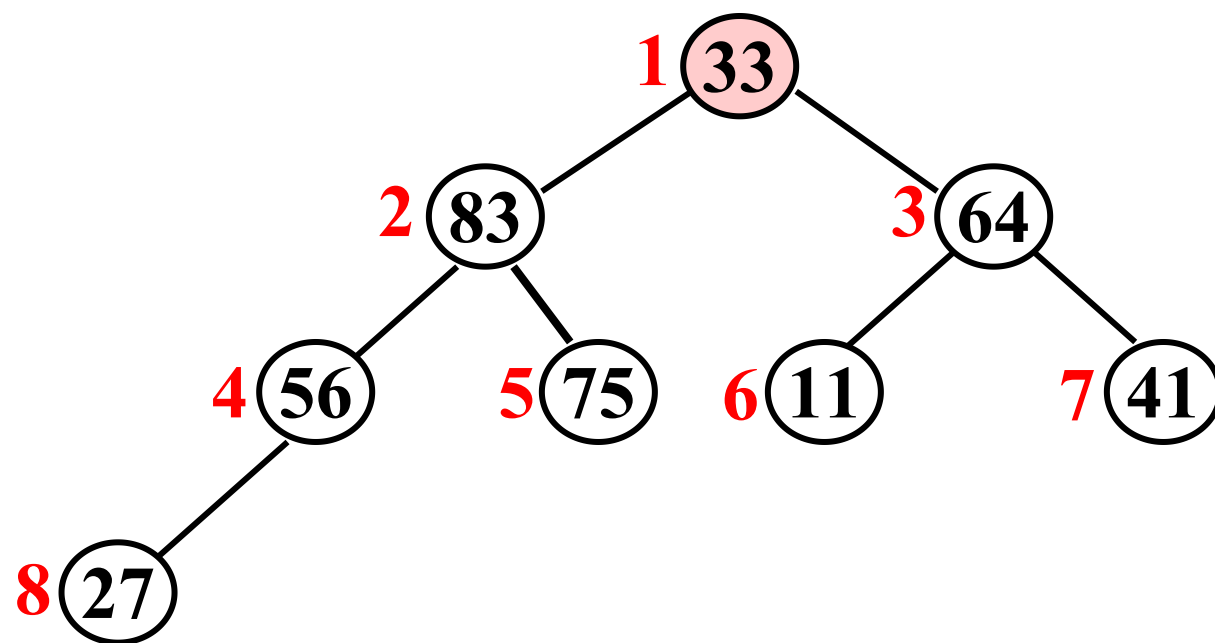
最大堆输出序列:





最大堆输出序列:

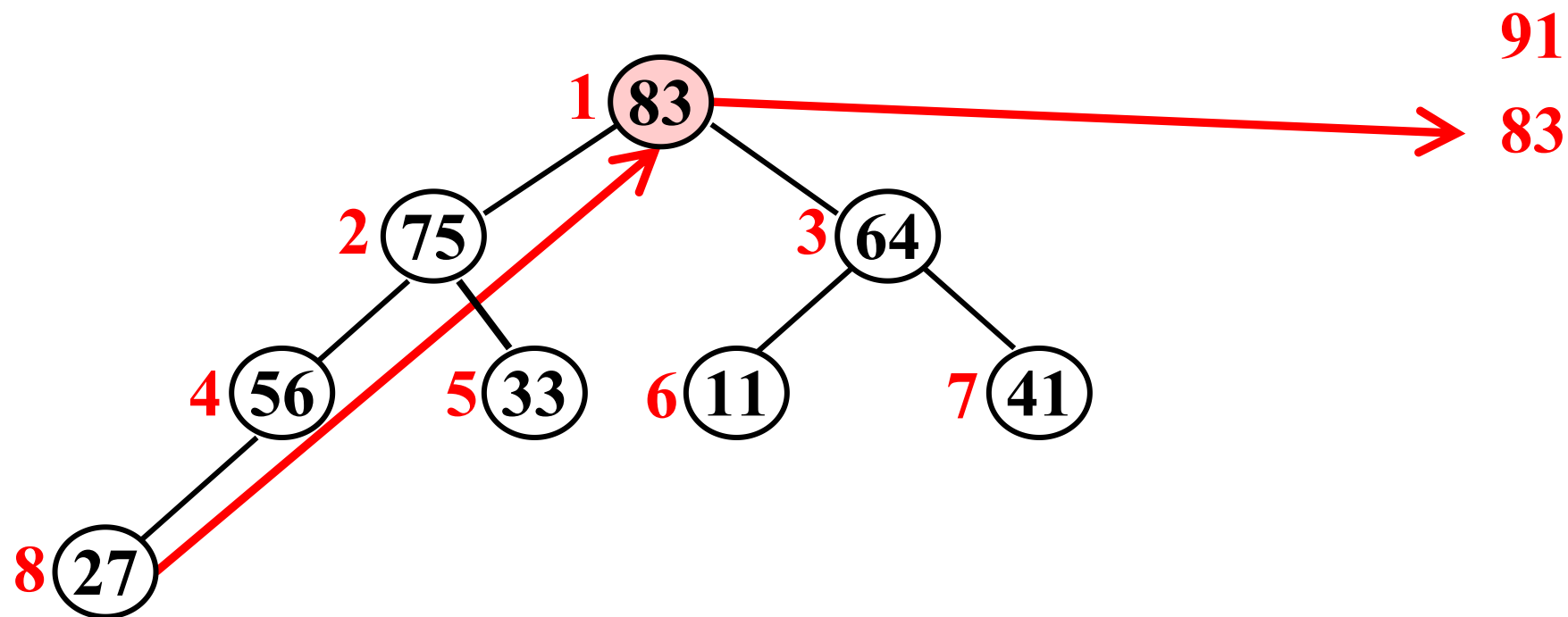
91



堆的输出



最大堆输出序列:



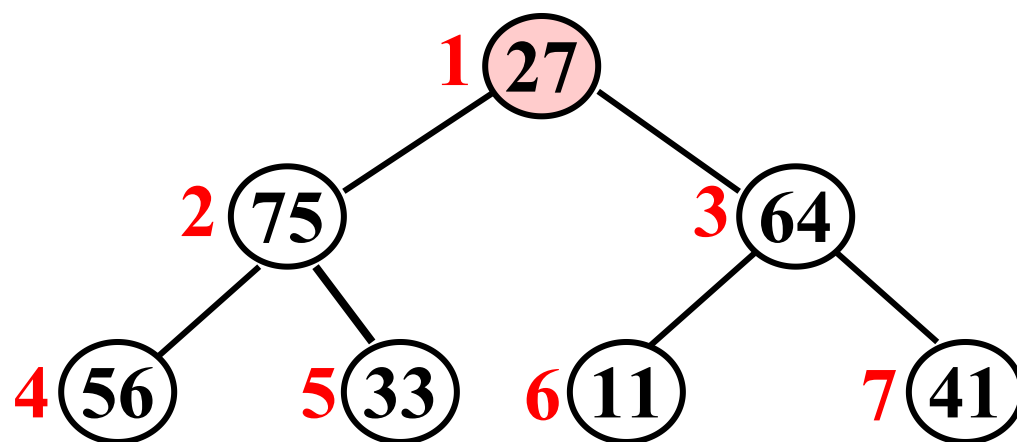
堆的输出



最大堆输出序列:

91

83



堆的输出

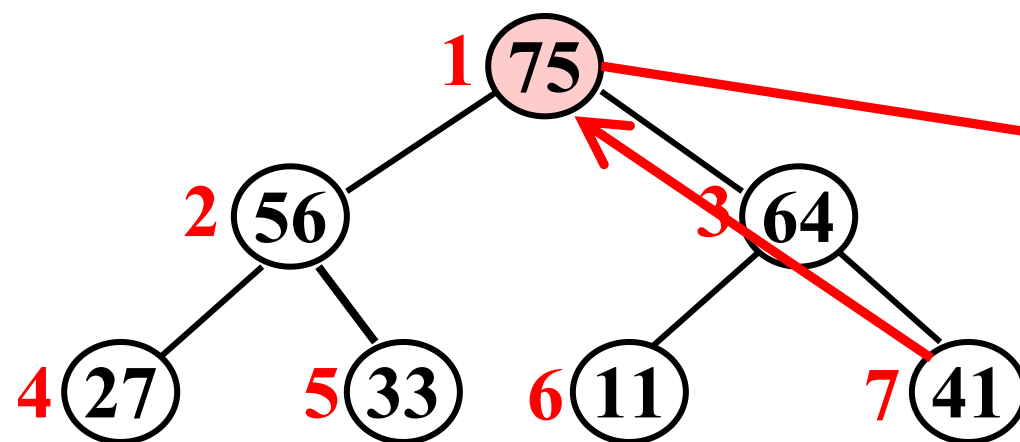


最大堆输出序列:

91

83

75



堆的输出

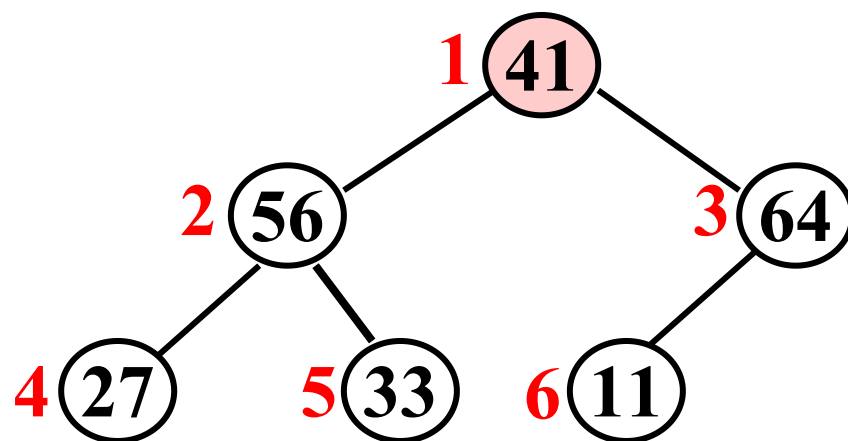


最大堆输出序列:

91

83

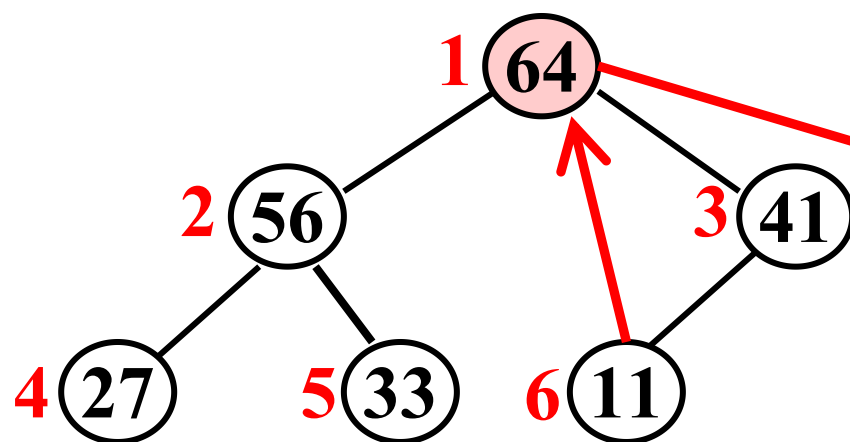
75



堆的输出



最大堆输出序列:



91

83

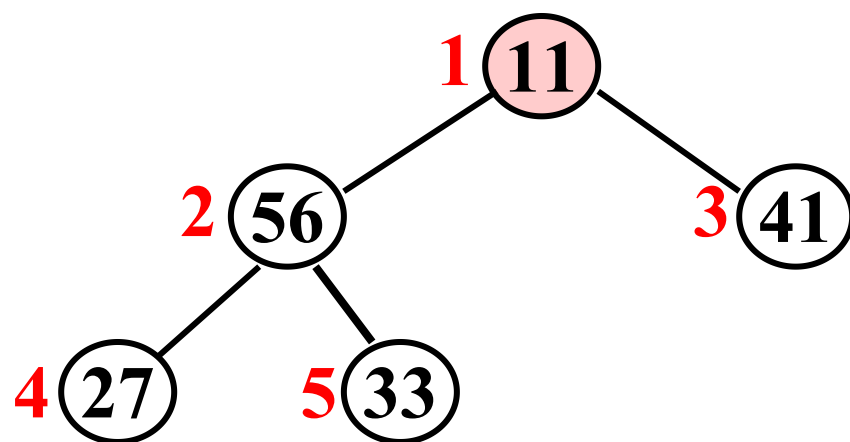
75

64

堆的输出



最大堆输出序列:



91

83

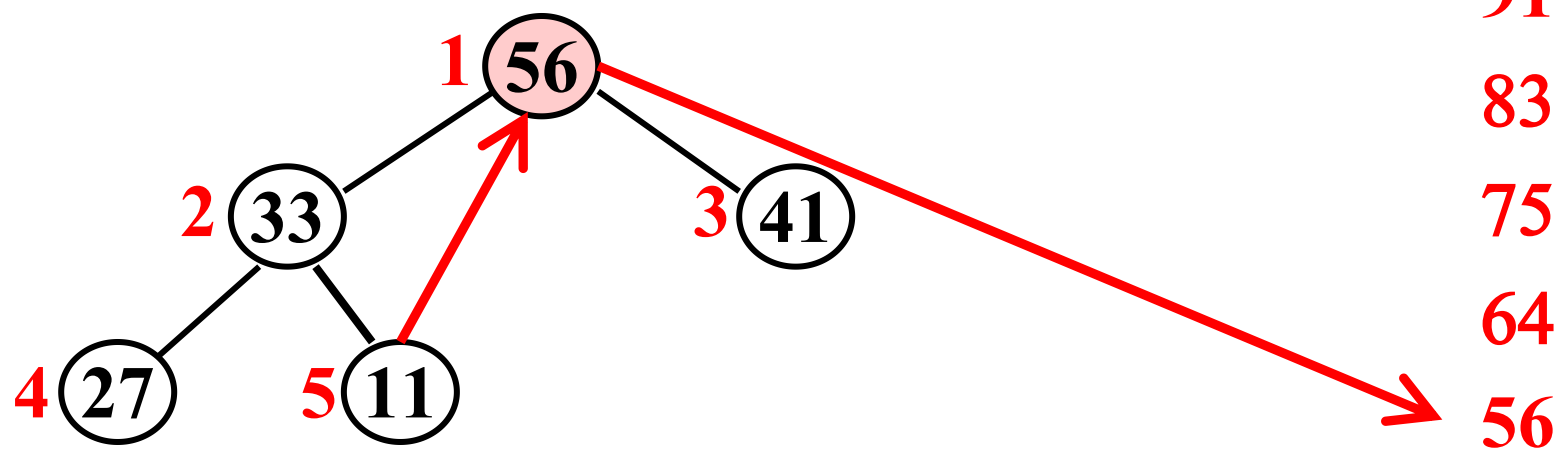
75

64

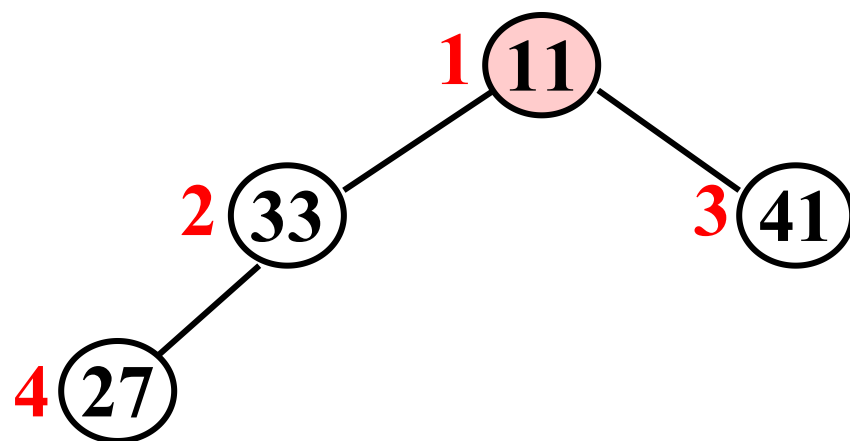
堆的输出



最大堆输出序列:



堆的输出



最大堆输出序列:

91

83

75

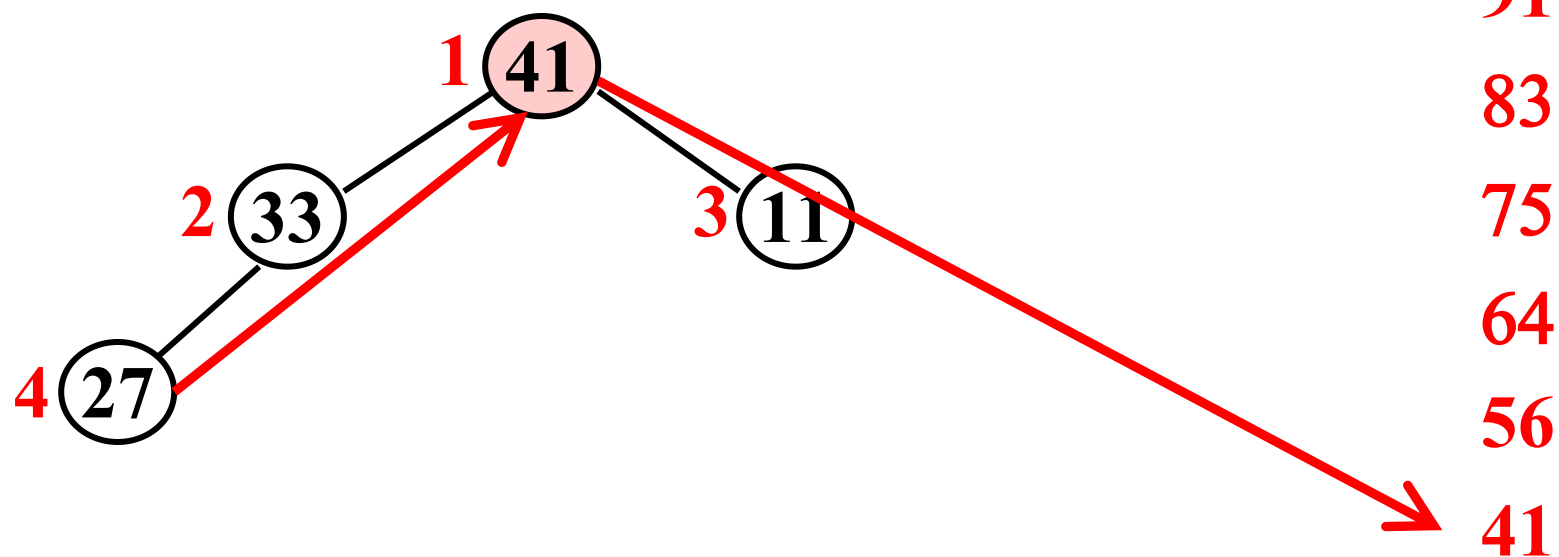
64

56

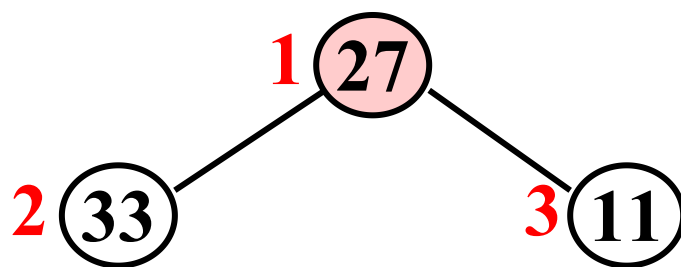
堆的输出



最大堆输出序列:



堆的输出



最大堆输出序列:

91

83

75

64

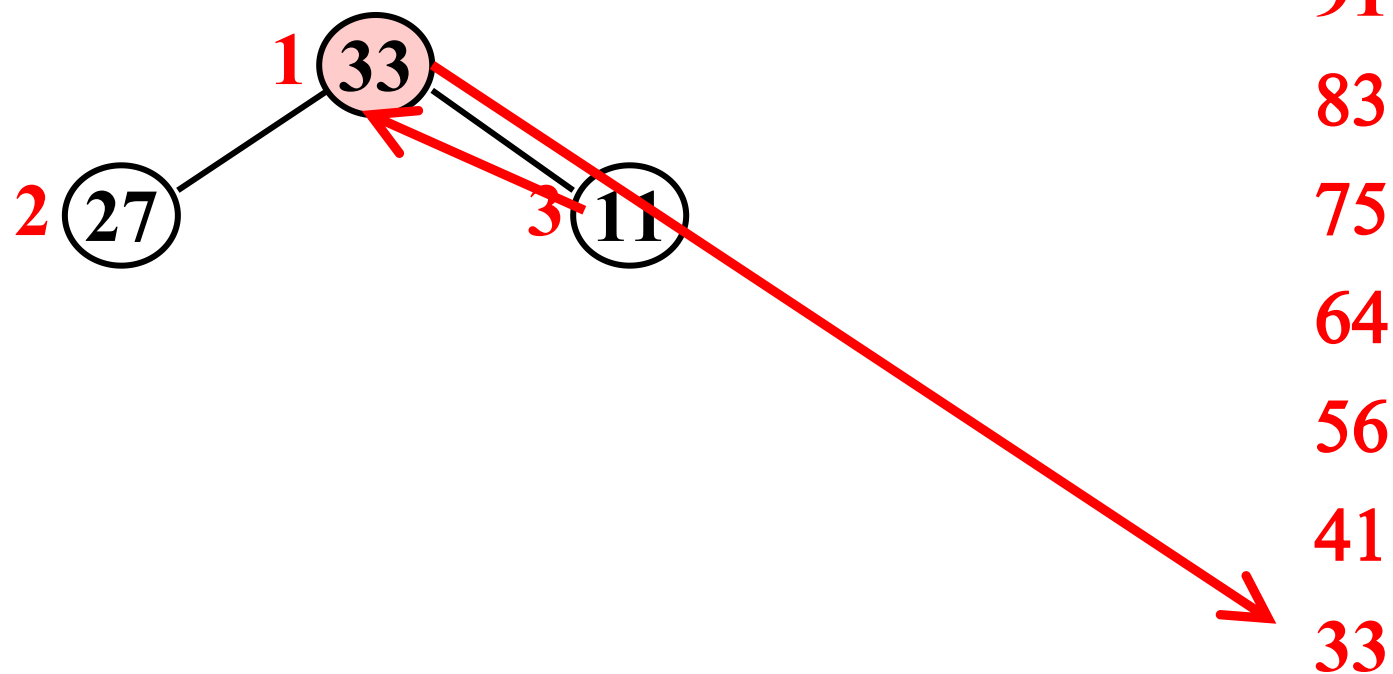
56

41

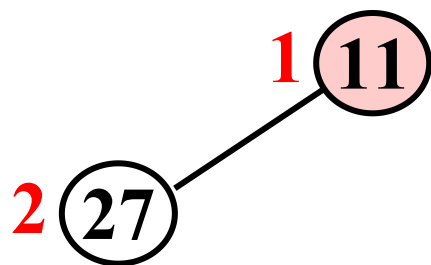
堆的输出



最大堆输出序列:



堆的输出



最大堆输出序列:

91

83

75

64

56

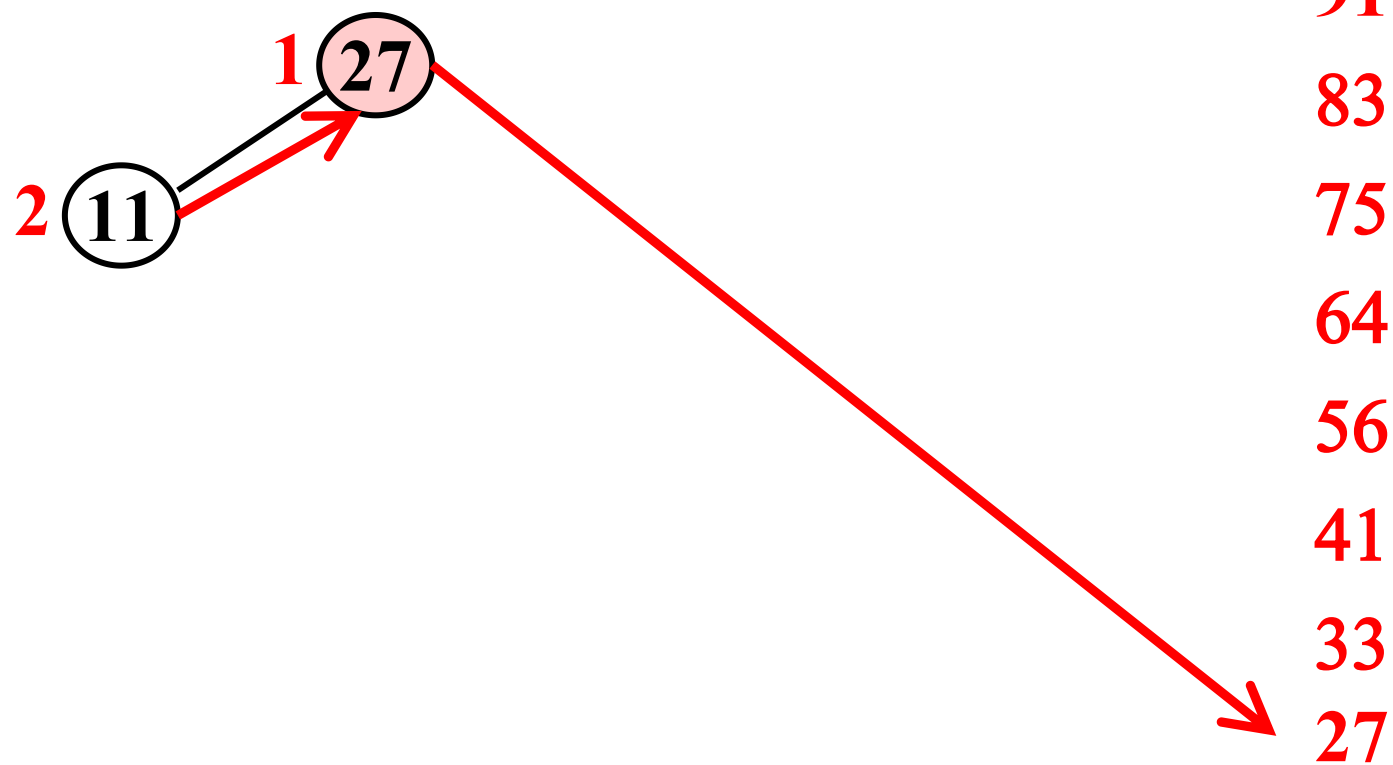
41

33

堆的输出



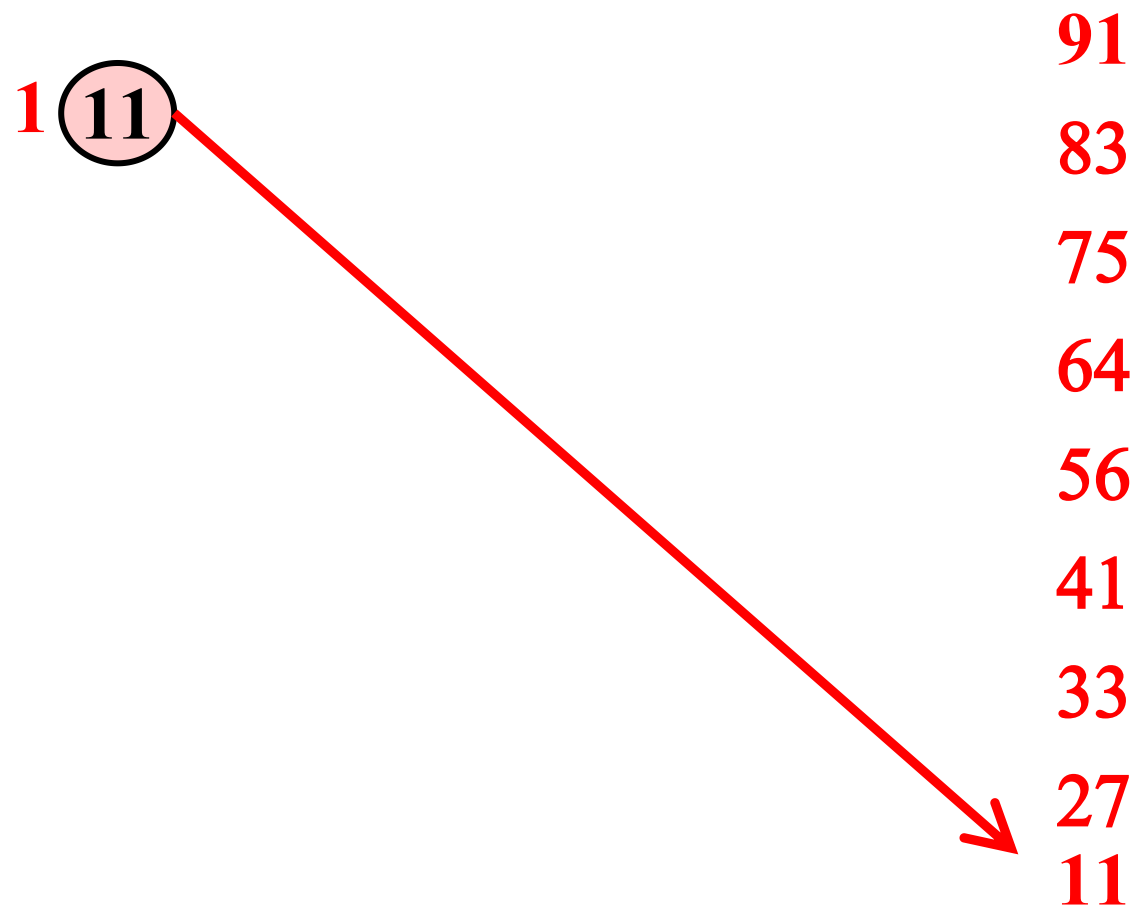
最大堆输出序列:



堆的输出



最大堆输出序列:



堆的输出



从堆中输出最大（或者最小）元素复杂度

最坏情况就是新的根自顶向下下沉到堆的叶子，于是时间代价为堆的深度 k

因此，从堆中输出最大（或者最小）元素时间代价为 $O(\log n)$

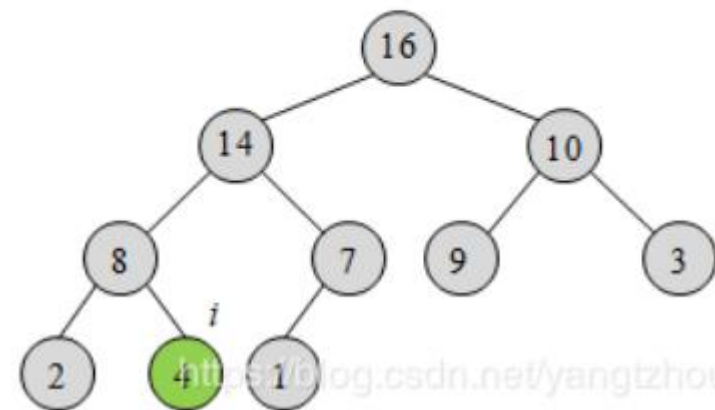
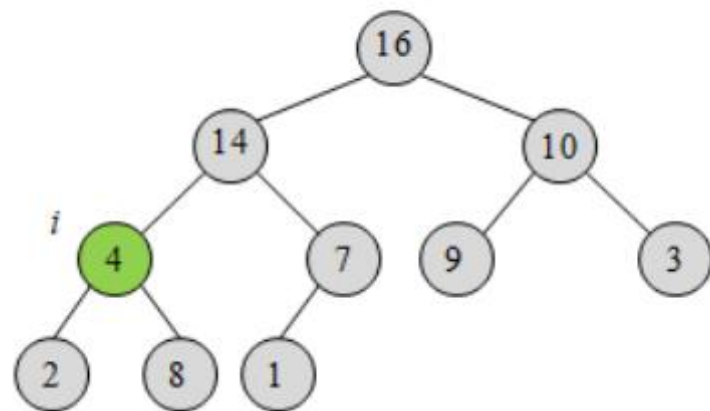
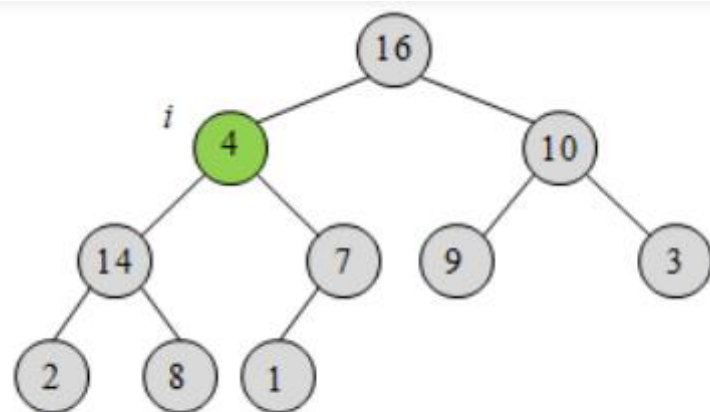
优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) $\text{Maximum}(S)$: 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) $\text{Extract_Max}(S)$: 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) **$\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。**

维护堆的性质



<https://blog.csdn.net/yangtzhou>



从堆中改变元素大小复杂度

最坏情况就是改变根然后自顶向下下沉到堆的叶子（或者改变叶子然后自底向上更新到根），于是时间代价为堆的深度 k
因此，从改变元素值的时间代价为 $O(\log n)$

建堆



建堆方法:

- ✓ 自底向上调整建堆
- ✓ 自顶向下插入建堆



建堆方法1：自底向上（迭代实现）

代码 6.3-1：构建最大堆

//参数A：一个用来构建最大堆的数组

```
1 BUILD-MAX-HEAP(A,i)
2   A.heap_size = A.length
3   r = RIGHT (i)
4   for i =  $\lfloor A.length/2 \rfloor$  downto 1
5     MAX-HEAPIFY(A,i)
```

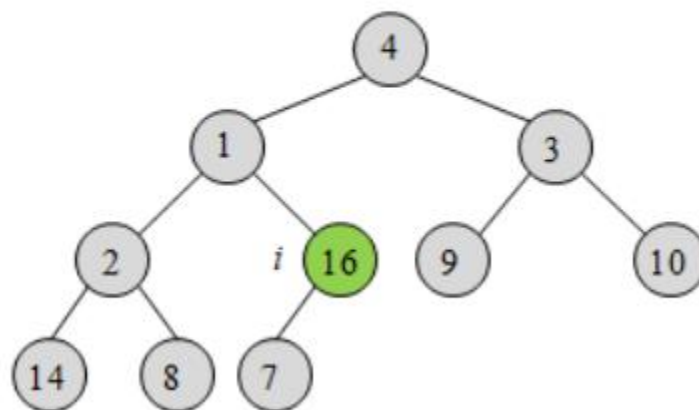
建堆



建堆方法1：自底向上（迭代实现）

原始数组 A

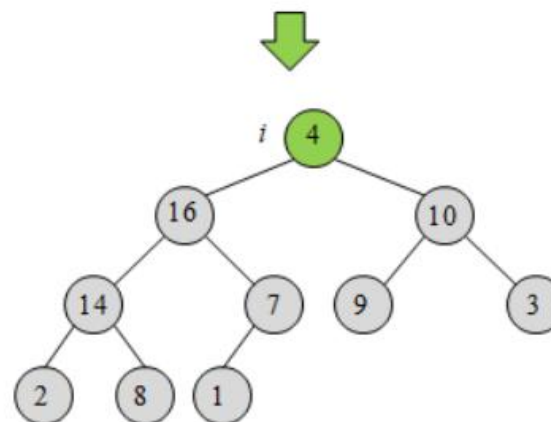
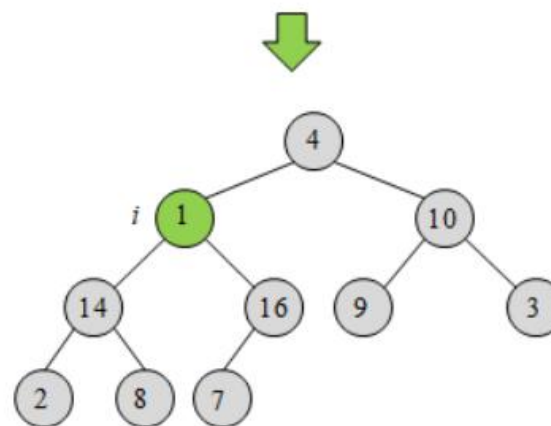
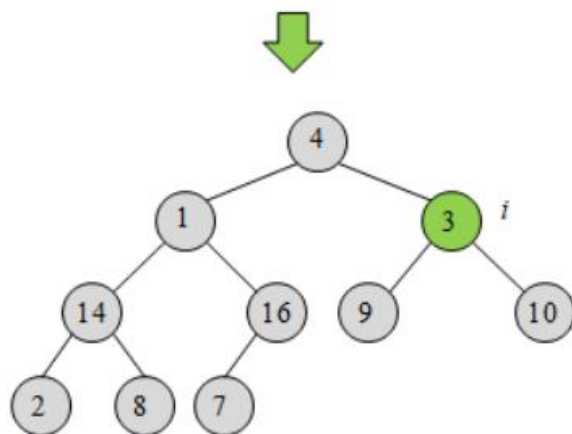
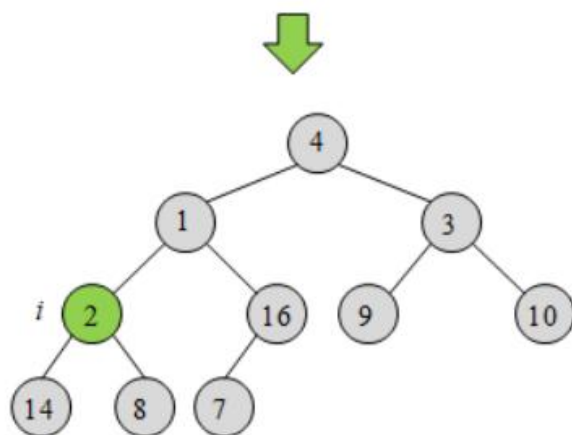
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



建堆



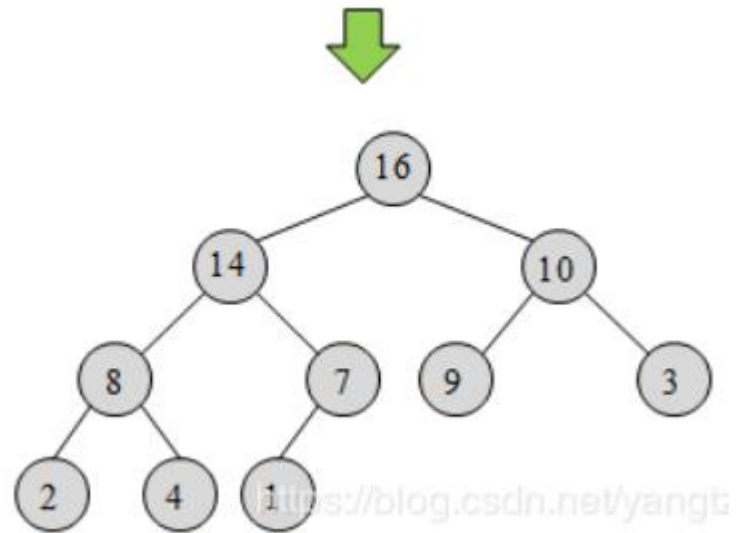
建堆方法1：自底向上（迭代实现）



建堆



建堆方法1：自底向上（迭代实现）





建堆方法2：插入建堆

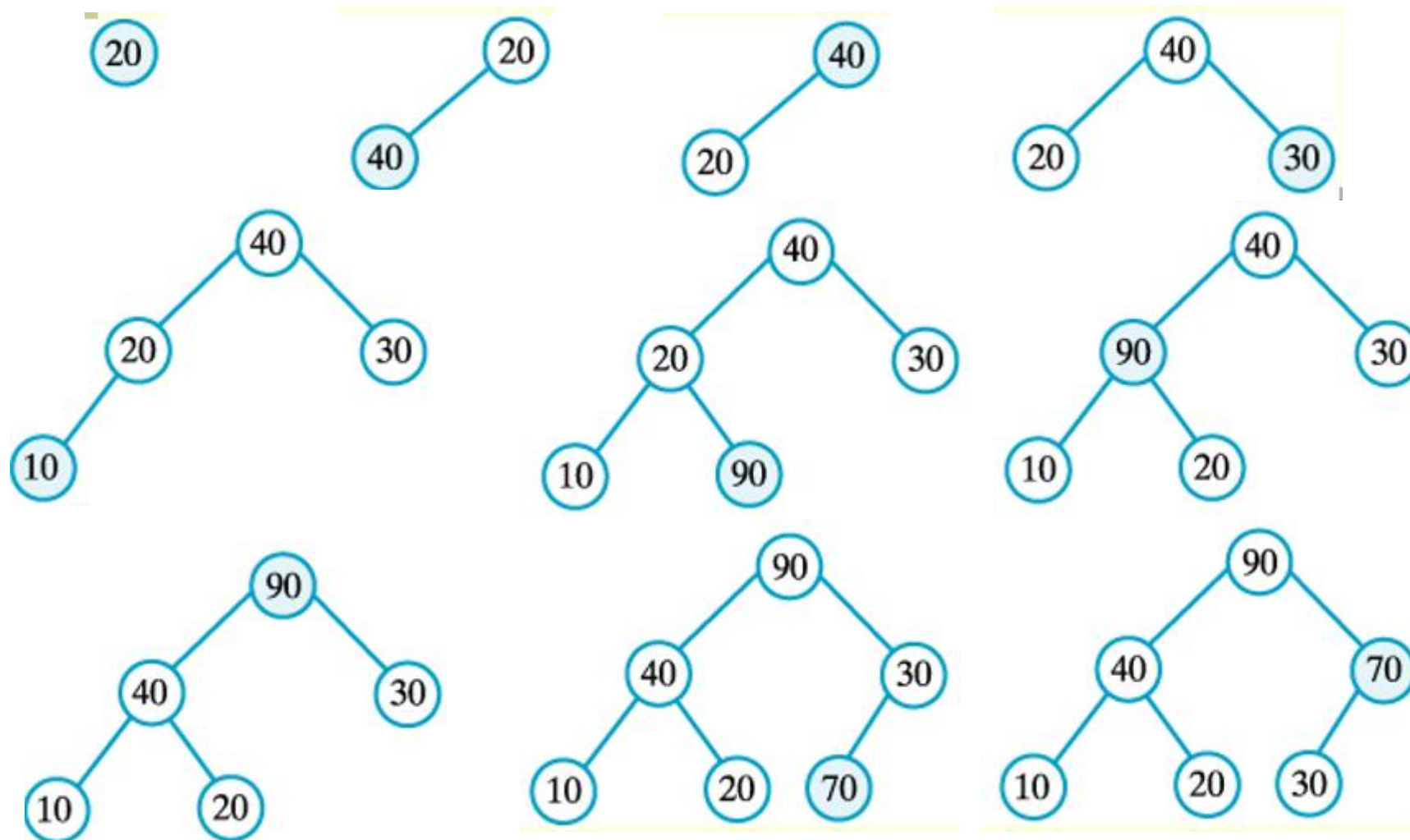
把一个数组看成两部分，左边是堆，右边是还没加入堆的元素，步骤如下：

- 1、数组里的第一个元素自然地是一个堆
- 2、然后从第二个元素开始，一个个地加入左边的堆，当然，每加入一个元素就破坏了左边元素堆的性质，得重新把它调整为堆。

建堆



建堆方法2:
插入建堆





建堆的复杂度

最坏情况就是，于是每个元素都是自底向上从叶子到根（或者自顶向下从根到叶子），每个元素调整代价为 $O(\log n)$

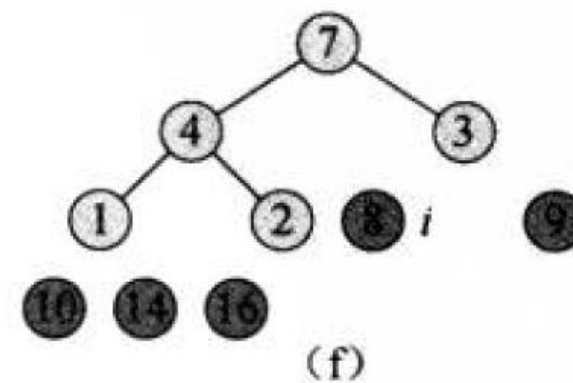
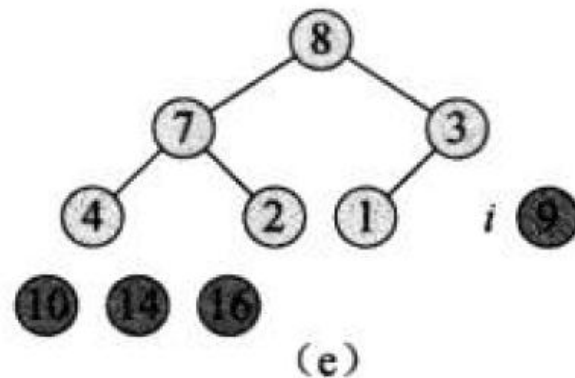
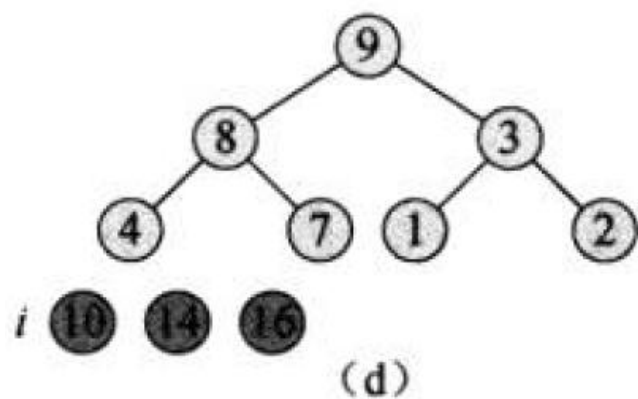
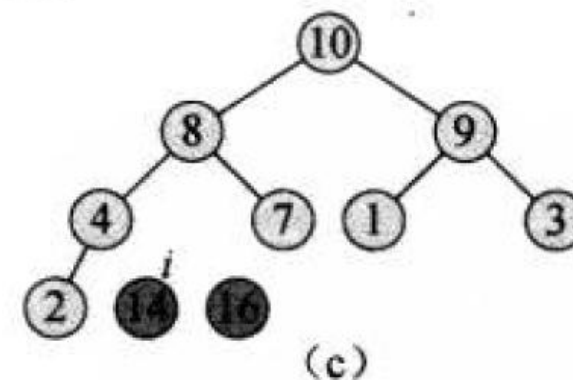
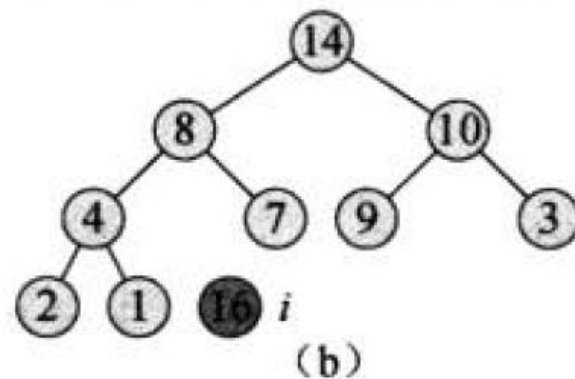
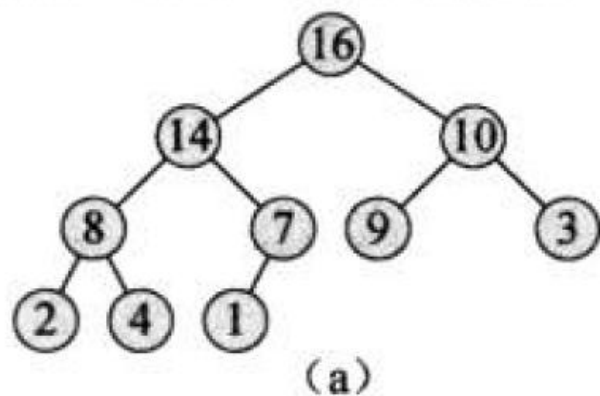
因此，对于 n 个元素，建堆时间代价为 $O(n \log n)$

堆排序算法



若在输出堆顶的最小值（最大值）后，使得剩余 $n-1$ 个元素的序列重又建成一个堆，则得到 n 个元素的次小值（次大值）.....如此反复，便能得到一个有序序列，这个过程称之为**堆排序算法**。

堆排序算法



堆排序算法



堆排序的时间复杂度

堆排序=堆调整 + 堆排序

堆排序时间复杂度=堆调整时间复杂度 + 堆排序时间复杂度

堆调整的时间复杂度为 $O(\log n)$ ，需要迭代 n 轮

总的时间复杂度为 $O(n \lg n)$

目录

第一节

堆

第二节

斐波拉契堆

回顾堆的定义



堆实质上是满足如下性质的完全二叉树：树中任一非叶结点的关键字均不大于(或不小于)其左右孩子(若存在)结点的关键字

对应序列上， n 个元素的序列 $\{k_1, k_2, \dots, k_n\}$ 当且仅当满足如下关系时，称之为**堆 (heap)**。若满足条件 $k_i \leq k_{i/2}$ 且则称**最大堆**；若满足条件 $k_i \geq k_{i/2}$ 则称**最小堆**

可合并堆



可合并堆 (Mergeable Heap) 是一种支持以下核心操作的抽象数据结构:

- ❑ **MAKE-HEAP()**: 创建空堆
- ❑ **INSERT(H, x)**: 插入元素
- ❑ **MINIMUM(H)**: 返回最小关键字 (或最大, 取决于堆类型)
- ❑ **EXTRACT-MIN(H)**: 删除并返回最小关键字
- ❑ **UNION(H₁, H₂)**: 将两个堆合并为一个, 并销毁原堆

在可合并堆的基础上, Prim算法还要求支持**DECREASE-KEY(H, x, k)**

(也就是将堆H中元素x赋值一个新的值k) 以及**DELETE(H, x)** (删除H中元素x) 。

二项堆



二项堆 (Binomial Heap) 其实是一种基于二项树构建的可合并堆数据结构。

二项树



- **二项树**是构建二项堆的基本单元，其结构呈递归形式，通常用 B_k 来表示度为 k 的二项树。
 - B_0 仅包含一个节点。
 - B_k ($k > 0$) 由两棵 B_{k-1} 树连接而成，其中一棵的根成为另一棵根的最左子节点。

二项树的形状



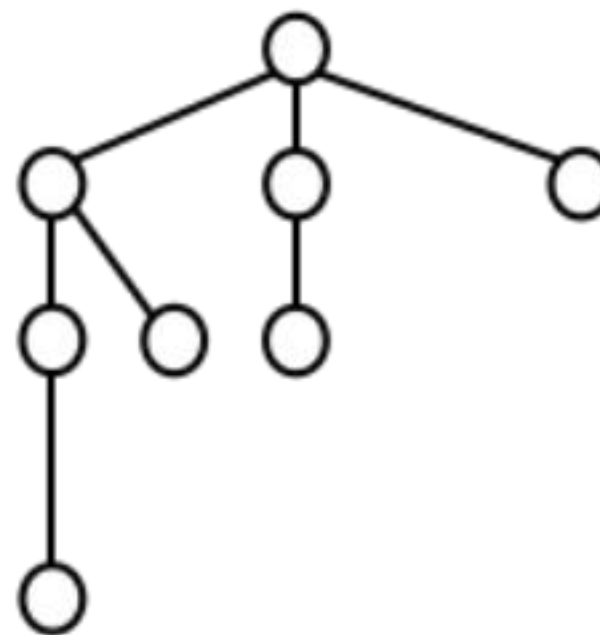
B_0



B_1



B_2



B_3

二项树性质



二项树具有节点数、高度、度数等方面的独特数学性质。

01

节点数量

一棵度为 k 的二项树 B_k 恰好包含 2^k 个节点。

02

树的高度

二项树 B_k 的高度为 k 。

03

根的度数

根节点的度数（子节点数量）为 k ，且其度数大于任何子节点的度数。

二项堆核心性质

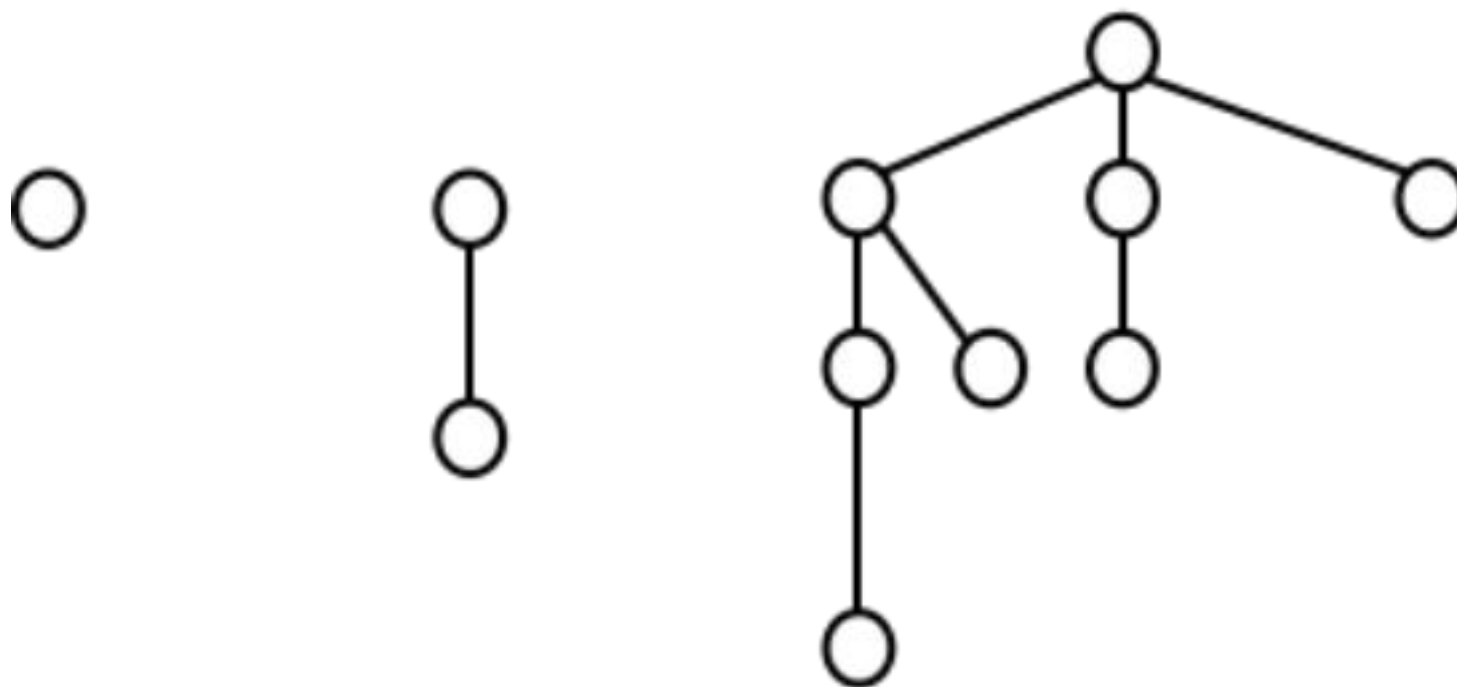


- 二项堆中二项树都是满足**最小堆性质**的，也就是二项树中每个元素比其孩子都小。
- 对于任意非负整数 k ，在同一个二项堆中，**最多只能存在一棵**度数为 k 的二项树 B_k 。

二项堆示例



□ 包含11个元素的堆



二项堆核心性质



□ 给定 n 个元素，二项树数量为 $O(\log n)$ ，二项树深度也为 $O(\log n)$

二项堆上操作——UNION

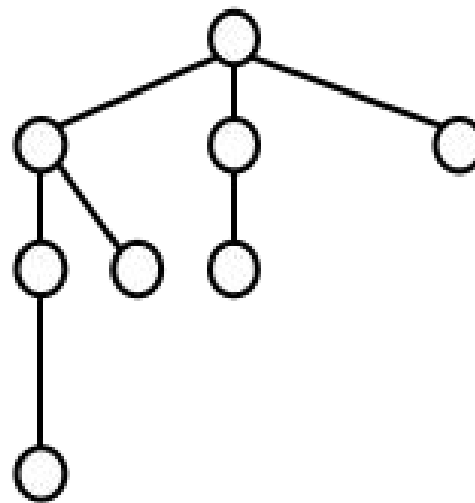
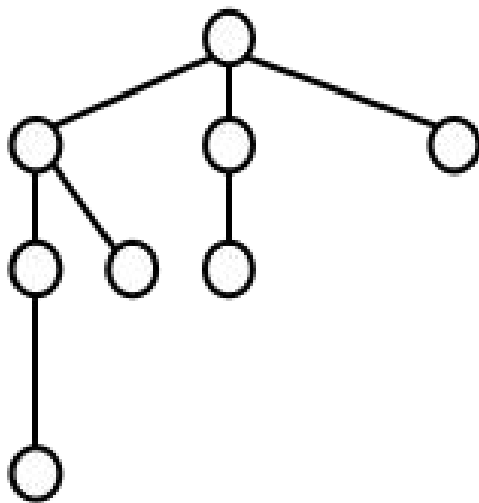


- 首先，我们将两个待合并的二项堆 H_1 和 H_2 中的所有二项树，按照它们的度数 k 从小到大进行排列。然后，像二进制加法一样“进位”合并相同度数的树。
- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有树，按阶排序（从小到大）。

二项堆上操作——UNION



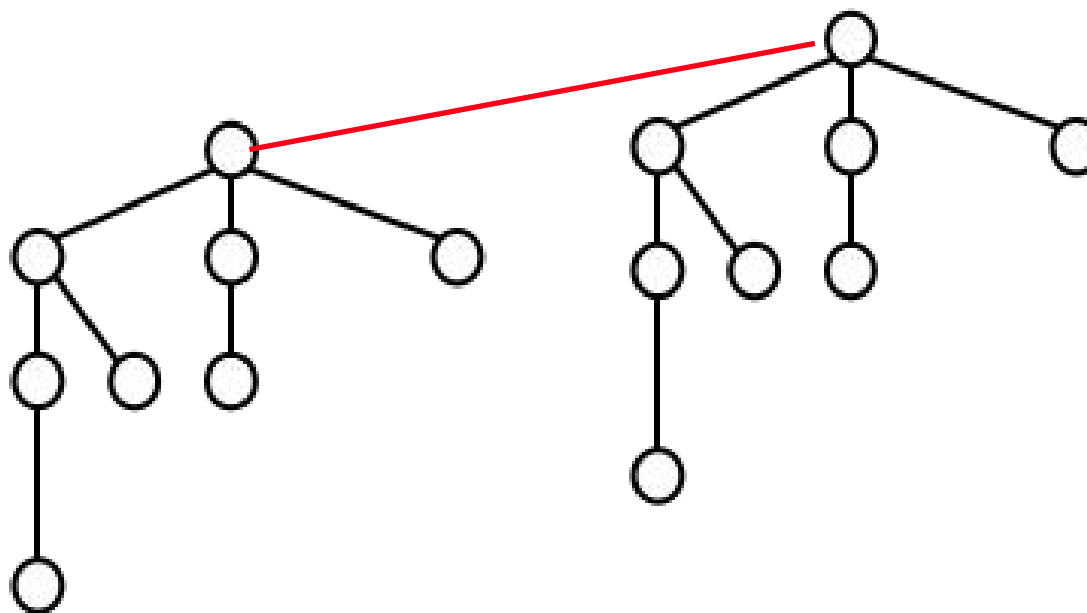
- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有二项树，按度数排序（从小到大）。
- 然后，两个度数都为 k 的二项树，可以合并成一个度数为 $k+1$ 的二项树。



二项堆上操作——UNION



- 首先，创建一个临时的“表” T ，包含 H_1 和 H_2 中所有二项树，按度数排序（从小到大）。
- 然后，两个度数都为 k 的二项树，可以合并成一个度数为 $k+1$ 的二项树。



代价 $O(1)$

二项堆上操作——UNION

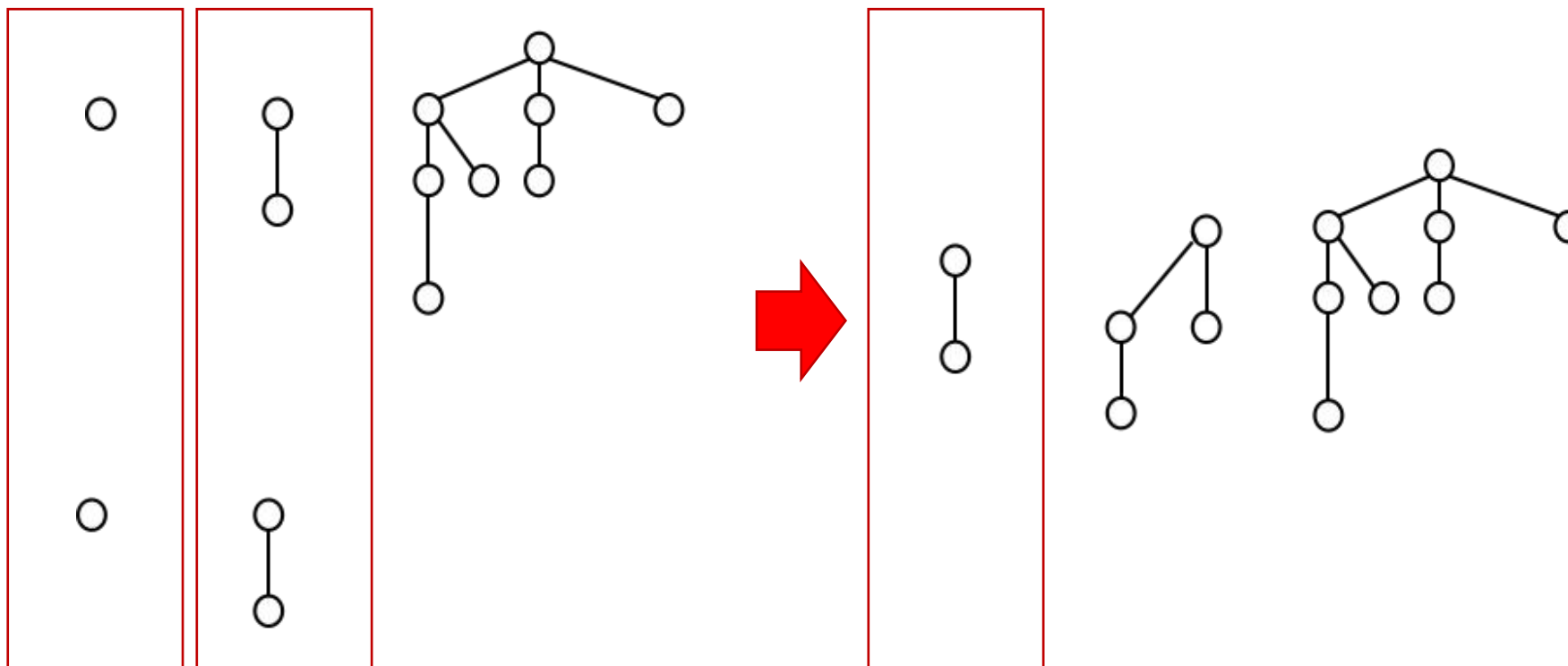


- 于是， H_1 和 H_2 中按度数从小到大合并二项树的时候，对于度数为 k 的情况有三种情况：
1. 度数为 k 的二项树 H_1 和 H_2 中都有1个，合并成1个度数为 $k+1$ 的二项树；
 2. 度数为 k 的二项树 H_1 或 H_2 中有1个，然后有2个度数为 $k-1$ 的二项树合并出来的1个度数为 k 的二项树，合并成1个度数为 $k+1$ 的二项树；
 3. 度数为 k 的二项树在 H_1 或 H_2 或度数为 $k-1$ 的二项树合并出来的度数为 k 的二项树里面只有1个，不操作；
 4. 度数为 k 的二项树没有，不操作。

二项堆合并示例



□ 以包含11个元素的堆和包含3个元素的堆合并为例



二项堆上操作——UNION

- 假设 H_1 和 H_2 中分别有 n_1 和 n_2 个元素，那么他们的二项树数量为 $O(\log n_1)$ 和 $O(\log n_2)$ 。于是， UNION代价 $O(\log n_1 + \log n_2)$

二项堆上其他操作

- ❑ **INSERT:** 等价于合并一个元素数量为1的堆，复杂度 **$O(\log n)$** ;
- ❑ **MINIMUM:** 在所有二项树的根中找最小，有 $O(\log n)$ 个二项树，所以复杂度 **$O(\log n)$** ;
- ❑ **EXTRACT-MIN:** 在所有二项树的根中找最小进行删除，会得到一个二项堆 H' ，合并原二项堆所剩下的二项树所组成的堆 H'' 和 H' ，复杂度 **$O(\log n)$** ;
- ❑ **DECREASE-KEY:** 将减少值堆元素在它所在二项树上往上升，复杂度 **$O(\log n)$** ;
- ❑ **DELETE:** 将该元素的键值降低到比当前堆中最小值还小，利用**DECREASE-KEY**，然后**EXTRACT-MIN**，复杂度 **$O(\log n)$**

二项堆改进

- ❑ 在二项堆基础上的重大改进，核心目标是降低某些关键操作的摊还时间复杂度，特别是那些在图算法（如 Dijkstra、Prim）中频繁调用的 **DECREASE-KEY** 和 **UNION**。
- ❑ 在斐波拉契堆里面，所有根用一个链表链起来，同时设定一个min指针指向最小值，所以**MINIMUM**代价是 $O(1)$
- ❑ 然后，相对于二项堆优化思路的核心在于：**懒惰 (Lazy) 操作**和**更宽松的树结构**

懒惰合并

□ 懒惰合并 (Lazy Merging)

- 二项堆: 合并时立即执行二进制加法, 逐阶链接同阶树, 代价 $O(\log n)$ 。
- 斐波那契堆: 合并两个堆时, 仅仅将它们的根链表拼接成一个链表, 不做任何树的合并, 因此 **UNIOIN** 操作只需 $O(1)$ 时间。

懒惰插入

- 懒惰插入

- 插入新元素时，直接将它作为一棵单节点树，添加到根链表中，而不尝试合并到现有树中。

- INSERT代价也是 $O(1)$ 。

更宽松的树结构

- 二项堆中每棵二项树是严格定义的：度数为 k 的树有固定形态，且根有 k 个子树（分别是 $B_0, B_1, \dots, B_{k-1}$ ）。
- 斐波那契堆中的树不要求根的子树按阶排序，只要**每个节点最多失去一个孩子**（通过标记机制维持）。

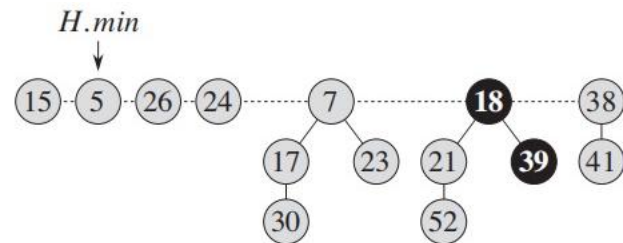
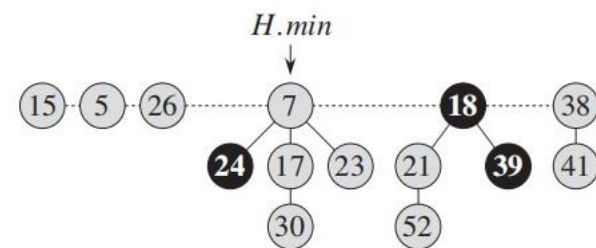
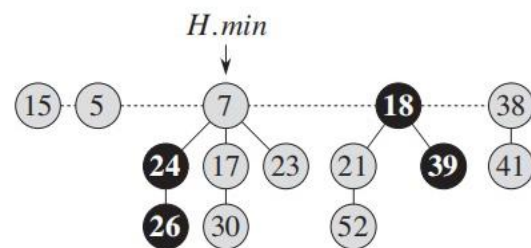
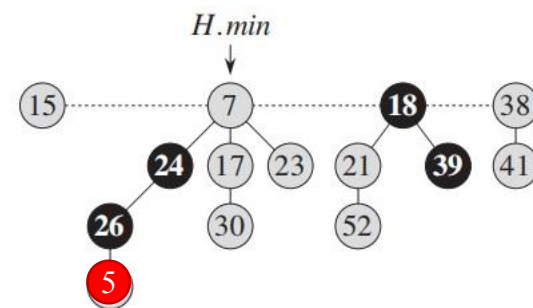
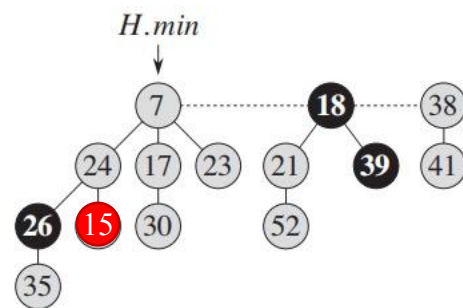
斐波拉契堆操作——DECREASE-KEY

级联切割

- ❑ **DECREASE-KEY**后，如果比父节点小，直接切断该节点与父节点的连接，并将该节点所在的子树整个移至根链表。
- ❑ 为了保持后续操作的摊还效率，对父节点做标记：如果父节点已经失去过一个孩子（标记为TRUE），则也将其切断并移至根链表，这个行为递归向上（级联切割）。
- ❑ 当一个元素被移至根链表，标记清为FALSE；一个元素初始状态标记都为FALSE

斐波拉契堆操作——DECREASE-KEY

示例



斐波拉契堆操作——DECREASE-KEY

代价分析

- 最坏情况下，单次操作的代价可能达到 $O(\log n)$ ；但是摊还代价为 $O(1)$

斐波拉契堆的DECREASE-KEY摊还分析

- 给定斐波拉契堆H，假设 $t(H)$ 是根链表中的树的数量， $m(H)$ 是被标记的节点数，那么定义势函数如下

$$\Phi(H) = t(H) + 2m(H)$$

- **DECREASE-KEY**每切断一个节点与其父节点连接，它为代价 $O(1)$ ，增加一棵树 ($t(H)$ 加 1) ，若切掉的节点原来有标记则 $m(H)$ 减 1 (标记被清除)
- 每次级联切割中，每切断一个节点与其父节点连接操作，要么减少一个标记节点 ($m(H)$ 减 1) ，要么将一个未标记节点变为标记 ($m(H)+1$)

斐波拉契堆的DECREASE-KEY摊还分析

- 假设斐波拉契堆 H 原本的势能为 $\Phi(H) = t(H) + 2m(H)$
- 一次**DECREASE-KEY**里做了 c 次切割，花了实际代价为 c ；
- 每次切割会增加一个树，所以**DECREASE-KEY**后的斐波拉契堆有 $t(H) + c$ 个树；第一次切割未必会减少标记，最后一次切割可能会增加一个标记，其他每次切割同时会消除一个标记，所以会 $m(H) - (c - 2)$

斐波拉契堆的DECREASE-KEY摊还分析

- 于是，势能变化为 $\Delta\Phi(H) = t(H) + c + 2(m(H) - c + 2) - (t(H) + 2m(H)) = 4 - c$
- 因此，一次DECREASE-KEY摊还代价为实际代价c加上势能变化4-c，也就是4，也就是O(1)

斐波拉契堆操作——EXTRACT-MIN

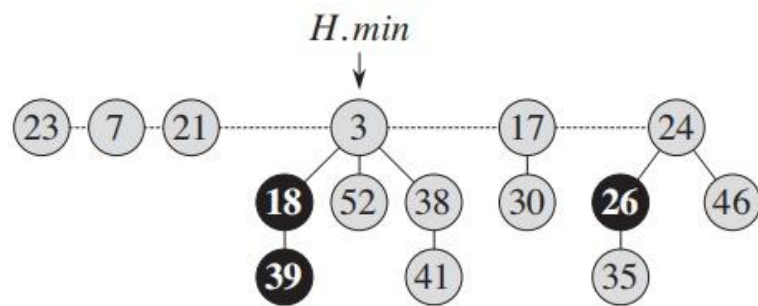
- 首先，利用根链表里面min指针，返回最小值并移除该节点；如果 min 有子节点，将所有子节点依次插入到根链表中（每个子节点成为一个新的根节点），并清空 min 的孩子列表；
- 由于之前的 insert、decrease-key 等操作都是“懒惰”的，根链表中可能有很多棵树（甚至 $O(n)$ 棵）。这一步的目标是将根链表中所有具有相同度数（degree）的树合并，直到每个度数最多只有一棵树。

斐波拉契堆操作——EXTRACT-MIN

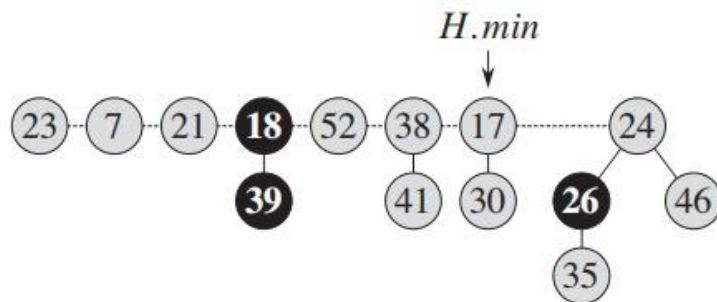
- 首先，利用根链表里面min指针，返回最小值并移除该节点；如果 min 有子节点，将所有子节点依次插入到根链表中（每个子节点成为一个新的根节点），并清空 min 的孩子列表；
- 由于之前的 insert、decrease-key 等操作都是“懒惰”的，根链表中可能有很多棵树（甚至 $O(n)$ 棵）。这一步的目标是将根链表中**所有具有相同度数的树合并**，直到每个度数最多只有一棵树。
- 树合并：使键值较小的成为新的根，另一个成为其孩子。

斐波拉契堆操作——EXTRACT-MIN

□ 初始状态:

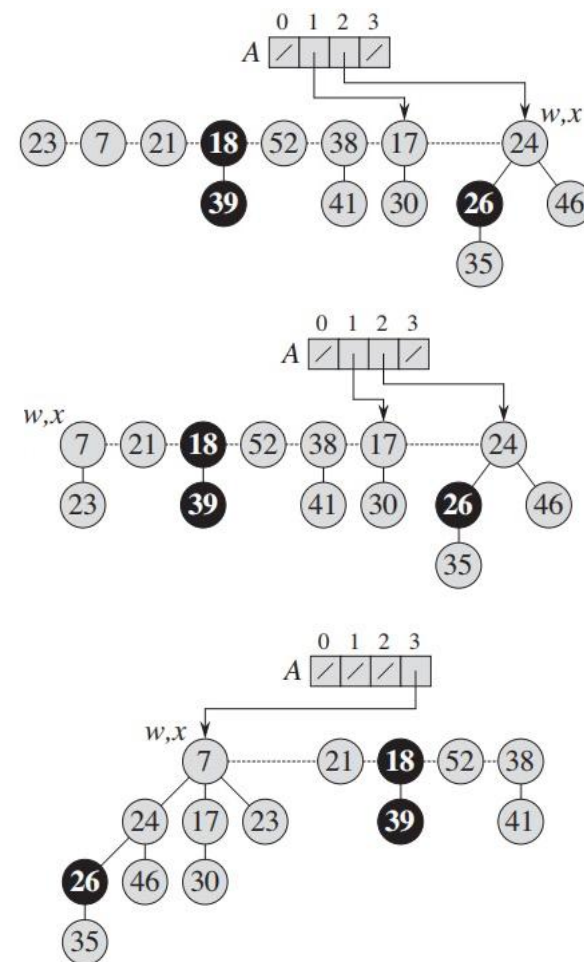
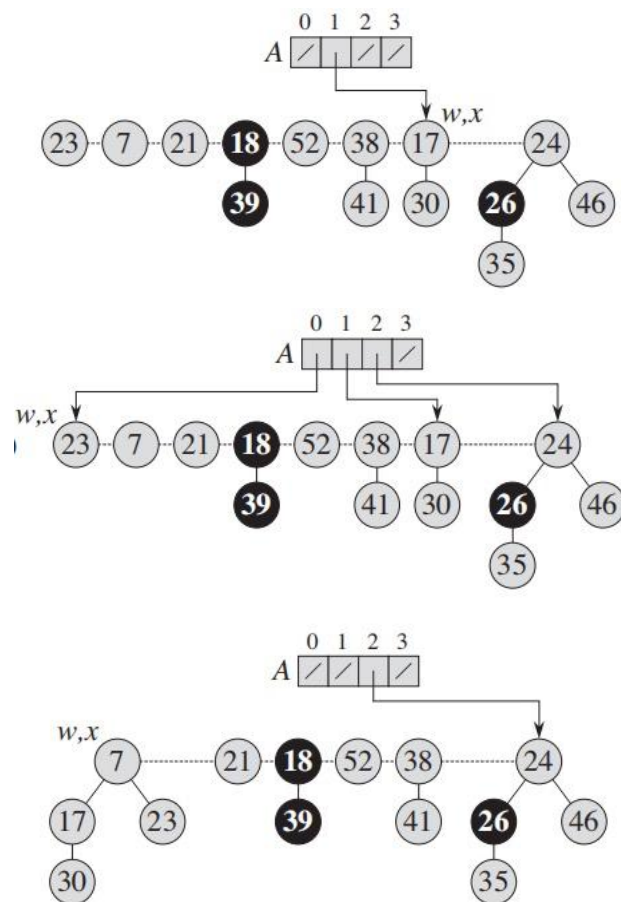


□ 返回并删去最小值:



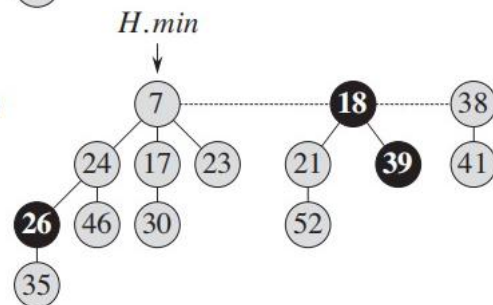
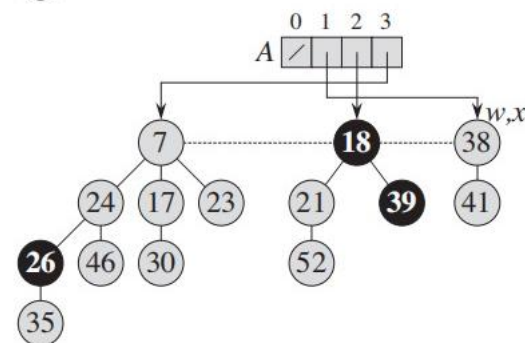
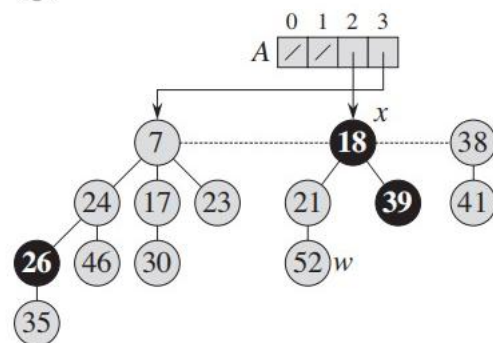
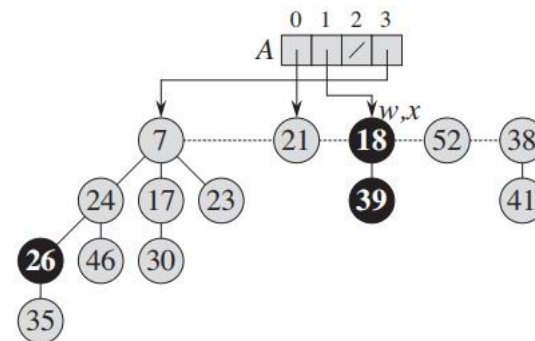
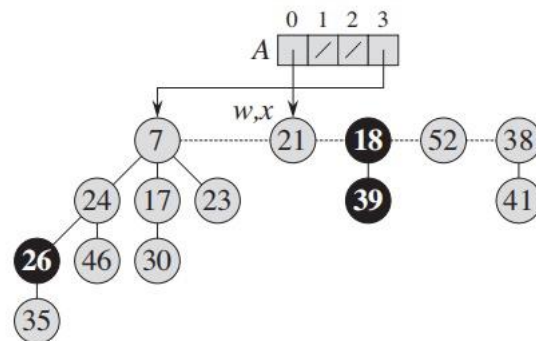
斐波拉契堆操作——EXTRACT-MIN

□ 树按照度数合并:



斐波拉契堆操作——EXTRACT-MIN

□ 树按照度数合并:



斐波拉契堆的EXTRACT-MIN摊还分析

- 可以证明，一个 n 个元素斐波拉契堆里面树度数上界 $D(n)=O(\log n)$ 。
- 那么**EXTRACT-MIN**过程前的 $t(H)$ 个合并到 $D(n)+1$ 个树上，实际代价会是 **$O(t(H) + D(n))$**
- **最坏情况：**若之前执行了大量 INSERT 和 DECREASE-KEY 操作而没有调用 EXTRACT-MIN，则根链表可能积累 $\Omega(n)$ 棵树（例如连续插入 n 个元素，每个成为独立一棵树）。此时合并步骤的代价为 $O(n)$ 。
- 因此 单次 EXTRACT-MIN 最坏时间复杂度为 **$O(n)$**

斐波拉契堆的EXTRACT-MIN摊还分析

- 给定斐波拉契堆H，假设 $t(H)$ 是根链表中的树的数量， $m(H)$ 是被标记的节点数，那么定义势函数如下

$$\Phi(H) = t(H) + 2m(H)$$

- 于是，**EXTRACT-MIN**过程后树的数量为 $D(n)+1$ ，同时**EXTRACT-MIN**过程不会增加标记点，所以**EXTRACT-MIN**过程后势能是 $D(n)+1+2m(H)$ ，所以势能变化为 $D(n)+1+2m(H)-t(H)-2m(H)=D(n)+1-t(H)$
- 所以**EXTRACT-MIN**摊还代价为 $O(t(H) + D(n))+D(n)+1-t(H)$ ，也就是 $O(D(n))=O(\log n)$

斐波拉契堆性质

- 在斐波那契堆中，一棵度数为 k 的树至少包含 F_{k+2} 个节点，其中 F 斐波那契数列 ($F_0=0$, $F_1=1$, $F_2=1$, $F_3=2$, ...)



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——